



Gonanku — Bahan Belajar & Persiapan Presentasi

Dokumen ini bukan dokumentasi teknis singkat — ini buku panduan lengkap.

Ditulis untuk dua tujuan:

1. **Bahan baca pribadi** — kamu (Sulthonika) bisa baca ulang dan ingat semua keputusan yang sudah diambil
2. **Bahan presentasi** — kalimat-kalimat di sini sudah disusun supaya bisa langsung kamu ucapkan di depan dosen (Bapak (dosen penguji)) tanpa perlu menerjemahkan jargon teknis

Aturan baca: ikuti urutan bab. Tiap bab nyambung ke bab berikutnya. Jangan loncat.



Daftar Isi

1. Cerita Singkat — Apa itu Gonanku?
 2. Kenapa Gonanku Dibutuhkan?
 3. Gambar Besar Arsitektur Sistem
 4. Pemilihan Stack Teknologi (dan Kenapa Bukan yang Lain)
 5. Setup Development di Laptop
 6. Database & Migrasi Schema (Alembic)
 7. Fitur Utama — Apa Saja & Bagaimana Cara Kerjanya
 - 7.11 ★ **CRUD Sistem — Operasi Lengkap (Bagian yang Diminta Dosen)**
 8. Security Hardening — Membuat Aplikasi Aman untuk Production
 9. ✨ Kegagalan & Cara Memperbaiki (Bagian PENTING untuk Presentasi)
 10. Deploy ke Cloud Run — Langkah Demi Langkah
 11. Penjelasan Command Deploy Baris-per-Baris
 12. 🗣️ Persiapan Presentasi ke Dosen
 13. ? Antisipasi Pertanyaan Dosen + Jawaban Siap Pakai
-

BAB 1: Cerita Singkat — Apa itu Gonanku?

Cerita yang Bisa Kamu Ucapkan ke Dosen

"Pak, kita semua punya masalah yang sama. Tahun 2019 saya foto wisuda kakak. Hari ini kalau Bapak minta saya tunjukkan foto itu, saya harus scroll galeri puluhan menit — mungkin malah tidak ketemu karena namanya `IMG_20190805_140312.jpg`, tidak ada petunjuk apa-apa. Sama dengan dokumen — saya sering lupa surat keterangan aktif kuliah semester 2 disimpan di folder mana.

Gonanku adalah jawaban dari masalah itu. Gonanku adalah **vault arsip pribadi yang cerdas**. Aplikasi web ini menyimpan, memahami, dan memanggil kembali kenangan saya — foto, dokumen, screenshot — lewat **AI**.

Saya tidak perlu mengetik judul, kategori, atau tag. AI yang baca isi file, kasih judul yang masuk akal, kategorikan, dan tandai. Saat saya butuh, saya tinggal tanya seperti ke teman: 'foto wisuda kakak tahun 2022' — Gonanku langsung menarik file yang relevan."

Nama & Filosofi

- **Gonanku** = "Goan-ku" → singkatan dari "**Goa**" (tempat penyimpanan tersembunyi) + akhiran "**-ku**" (milik saya). Filosofinya: *vault pribadi yang dalam dan aman*.
- "**Personal AI Memory Vault**" = slogannya.

Apa yang Bukan Gonanku?

- Bukan Google Drive (Drive cuma simpan file, tidak paham isinya)
- Bukan iCloud Photos (Photos paham foto, tapi tidak paham dokumen)
- Bukan ChatGPT (ChatGPT pintar tapi tidak tahu isi vault saya)

Gonanku = gabungan: **penyimpanan + pemahaman + pencarian natural**.

BAB 2: Kenapa Gonanku Dibutuhkan?

3 Masalah Inti yang Diselesaikan

Masalah #1: File punya nama yang tidak bermakna

- Galeri kamu penuh `IMG_2389.jpg`, `Screenshot_20240515.png`, `WhatsApp Image 2024-06-12.jpeg`
- Tidak ada cara cari kecuali scroll satu-satu
- **Gonanku solve:** AI baca isi gambar pakai vision model, kasih judul deskriptif otomatis seperti *"Foto Wisuda Kakak Sarjana Teknik di Telkom University Surabaya, Agustus 2022"*

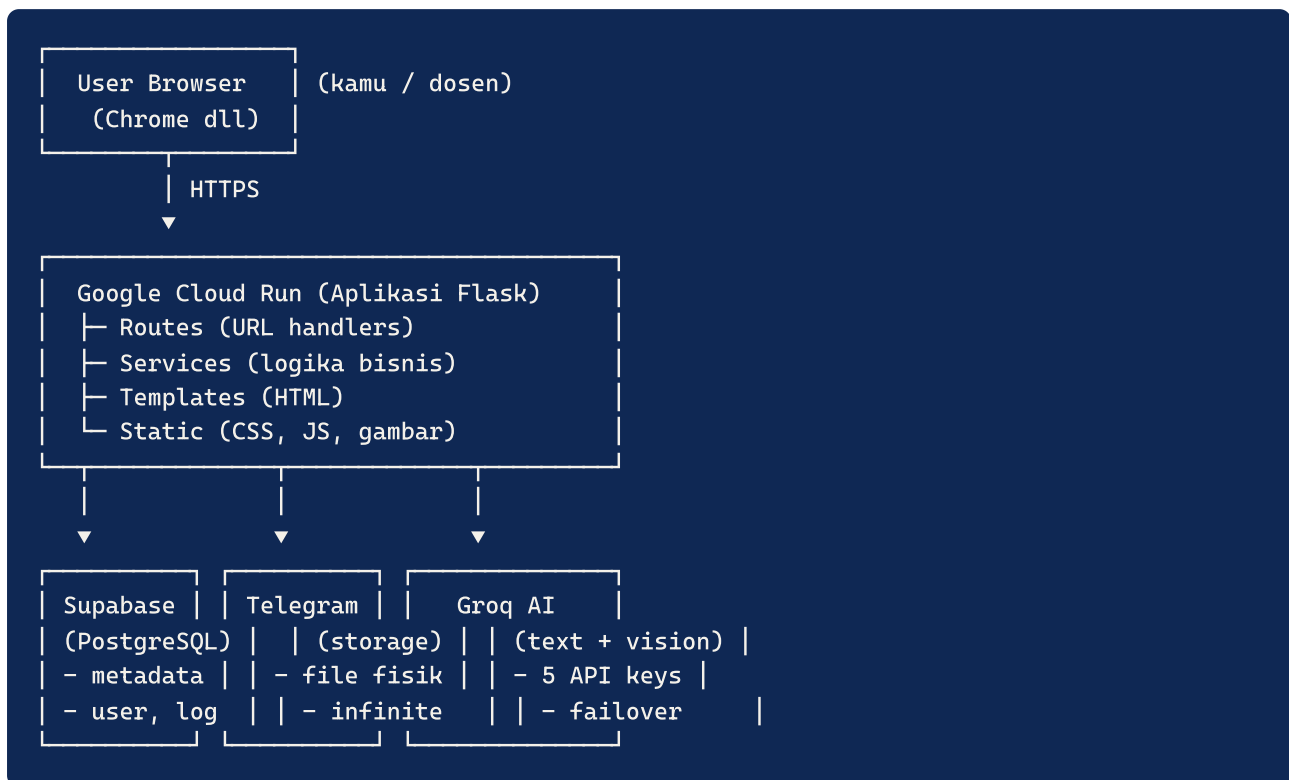
Masalah #2: Pencarian harus pakai kata kunci tepat

- Cari di Drive: harus inget nama file. Salah satu huruf → tidak ketemu.
- **Gonanku solve:** chatbot pakai bahasa biasa. Cukup tanya *"foto ulang tahun adik tahun lalu"* — sistem rangkai ulang konteksnya pakai AI

Masalah #3: Privasi

- File pribadi di Google Drive bisa di-share, di-index, dipakai untuk train AI mereka
 - **Gonanku solve:** file fisik di-store di **Telegram private channel** kamu sendiri (akses pribadi), metadata di DB yang full kontrol kamu (Supabase). Isolasi data per akun ketat — User A tidak akan pernah lihat file User B.
-

BAB 3: Gambar Besar Arsitektur Sistem



Cara Membaca Diagram di Atas (untuk Dosen)

"Pak, kalau ada user upload foto, file foto-nya **tidak disimpan di Cloud Run**. Cloud Run cuma jadi 'jembatan/penerima sementara'. File-nya langsung dilempar ke **private channel Telegram** (storage gratis, kapasitas tidak terbatas). Sedangkan **informasi tentang file-nya** — judul, kategori, tag, tanggal momen, ringkasan AI — disimpan di **Supabase PostgreSQL**.

Jadi tiga layanan bekerja sama dengan tugas jelas: **Cloud Run = otak/server aplikasi**, **Telegram = gudang file fisik**, **Supabase = gudang informasi/metadata**. Ini namanya arsitektur **storage hybrid** — kombinasi storage berbeda sesuai jenis data."

BAB 4: Pemilihan Stack Teknologi

Ini adalah **bagian paling penting untuk presentasi** — dosen pasti tanya "kenapa pakai ini, bukan itu". Saya susun jawaban siap pakai.

4.1 Backend: Flask (Python)

Pilihan	Status	Alasan
Flask ✓	DIPILIH	Ringan, simpel, app factory pattern, ecosystem mature, cocok untuk solo developer
Django	✗	Terlalu besar, banyak fitur tidak terpakai (CMS, admin built-in), startup time lambat
FastAPI	✗	Lebih cocok untuk pure API, butuh setup template engine terpisah. Gonanku punya UI HTML jadi Flask + Jinja2 lebih ringkas

Kalimat untuk dosen:

"Saya pilih Flask karena saya butuh framework yang fleksibel tapi tidak over-engineered. Django terlalu opinionated untuk skala project ini. FastAPI bagus untuk pure API tapi Gonanku punya halaman HTML, jadi Flask + Jinja2 lebih pas. Plus, ekosistem extension Flask (Flask-Login, Flask-WTF, Flask-Migrate) sudah mature."

4.2 Database: Supabase PostgreSQL — KENAPA BUKAN CLOUD SQL?

Ini pertanyaan dosen yang paling mungkin keluar. Jawaban detail:

Perbandingan Harga (per Juni 2026)

Layanan	Harga termurah/bulan	Catatan
Supabase Free tier 	Rp 0	500 MB DB, 50 MB file storage, 5 GB bandwidth, unlimited API requests
Cloud SQL (db-f1-micro) 	~Rp 165.000 (~\$10)	Sudah termurah. Tetap dicharge meski 0 query
Cloud SQL (db-g1-small) 	~Rp 400.000 (~\$25)	Mid-tier
Cloud Spanner 	~Rp 1.500.000+	Enterprise-only

Kalimat untuk dosen:

"Pak, ini project kuliah dengan budget Rp 0. Saya butuh database yang gratis selamanya. Supabase memberikan PostgreSQL managed — artinya tidak perlu setup server sendiri — dengan **free tier permanen**, kapasitas 500 MB. Untuk catatan tentang file (judul, kategori, tag), 500 MB cukup untuk metadata ratusan ribu file.

Bandingkan dengan Cloud SQL — yang juga produk GCP — paling murah **Rp 165.000 per bulan walaupun nol user yang akses**. Itu fixed cost, tetap dicharge.

Saya cek tagihan saya 6 hari pertama deploy: total cuma **Rp 1.029**. Itu pun bukan biaya DB (DB-nya di Supabase free), tapi biaya Cloud Run dan transfer data. Setahun ekstrapolasi ~Rp 60.000 — lebih murah dari kopi sehari."

Bonus: Cloud SQL sebenarnya ada free tier?

TIDAK. Cloud SQL hanya kasih **\$300 credit selama 90 hari pertama** untuk akun baru GCP. Setelah itu langsung dibilling. Supabase free tier **selamanya** (asal di bawah limit).

Trade-off Supabase yang Honest:

Plus Supabase	Minus Supabase
Free tier permanen	Cold start kadang 3-5 detik (queries pertama setelah idle)
Setup 10 menit	Limit 60 connection bersamaan (free tier) → solved dengan pooler
Built-in pgvector untuk future semantic search	Vendor lock-in (DB di Supabase, app di GCP — agak split)
Dashboard UI bagus untuk debug	Region terdekat Singapore (latency ~30ms dari Surabaya)

4.3 Storage File: Telegram Private Channel — KREATIF & GRATIS

Kenapa Telegram?

Pilihan	Harga	Limit
Telegram Bot API 	Rp 0 selamanya	50 MB per file via Bot API, tidak ada limit total
Google Cloud Storage	~\$0.020/GB/bulan	Pay-per-use. Setelah free tier 5 GB pertama habis
AWS S3	~\$0.023/GB/bulan	Sama
Cloudinary	\$89/bulan untuk Plus	Bisa free tier 25 GB

Kalimat untuk dosen:

"Pak, ini bagian paling kreatif dari Gonanku. File fisik — foto, dokumen, screenshot — semua dikirim ke **private channel Telegram** saya lewat Bot API.

Kenapa Telegram? Pertama, Telegram secara **resmi tidak ada batas total storage** untuk channel — boleh upload terus selamanya. Kedua, setiap file otomatis dapat URL CDN (Content Delivery Network) gratis — artinya bisa diakses cepat dari mana saja.

Logikanya begini, Pak: Telegram sudah punya infrastruktur penyimpanan file kelas dunia, dan mereka berikan gratis untuk pengguna channel. Saya memanfaatkan ini sebagai 'storage backend' aplikasi saya. Trade-off-nya: maksimal 50 MB per file via Bot API. Untuk vault pribadi (foto + dokumen), itu lebih dari cukup — kebanyakan file kita di bawah 10 MB."

Aliran upload (untuk dijelaskan ke dosen):

1. User pilih foto di browser
2. Browser kirim ke Cloud Run via HTTPS
3. Cloud Run validasi (ekstensi, ukuran)
4. Cloud Run kirim file ke Telegram Bot API
5. Telegram balas dengan file_id + chat_id + message_id
6. Cloud Run simpan referensi (file_id, chat_id) ke Supabase
7. AI Groq baca isi file → generate metadata
8. Metadata disimpan ke Supabase
9. File temporer di Cloud Run dihapus (filesystem ephemeral)
10. User lihat hasil di Dashboard

4.4 AI: Groq — KENAPA BUKAN OPENAI/CLAUDE?

Layanan	Harga	Speed	Free tier
Groq 	Free tier generous	300-500 tokens/sec (super cepat)	Ya, 5 API keys @ 14400 req/day = 72000 req/day total
OpenAI GPT-4o	\$2.5 per 1M token input	~80 tokens/sec	Tidak ada (kecuali \$5 credit awal)
Claude 3.5 Sonnet	\$3 per 1M token input	~70 tokens/sec	Tidak ada
Gemini Flash	Free tier ada tapi terbatas	~150 tokens/sec	Ya tapi 15 req/menit

Kalimat untuk dosen:

"Groq itu provider AI inference yang khususnya cepat — pakai chip khusus bernama LPU (Language Processing Unit). Model yang saya pakai adalah Llama 3.3 70B (open source dari Meta) untuk teks, dan Llama 4 Scout untuk vision.

Yang penting: **Groq kasih free tier yang generous**. Setiap API key dapat 14.400 request per hari. Saya pakai 5 key, total 72.000 request per hari. Untuk vault pribadi — yang upload-nya paling 50-100 file per bulan — ini lebih dari cukup. Kalau pakai GPT-4o atau Claude, sekali upload bisa \$0.05, sebulan bisa \$5-10. Gonanku target gratis."

Pembagian beban 5 API keys (failover strategy):

Key 1 → Metadata teks (judul, kategori, tag, ringkasan)
 Key 2 → Chatbot (intent parser + jawaban)
 Key 3 → Vision (foto-to-text + deskripsi visual)
 Key 4 & 5 → Cadangan / fallback

Kalau Key utama kena rate limit (HTTP 429) → otomatis coba Key cadangan.

4.5 Deploy: Cloud Run — KENAPA BUKAN APP ENGINE / VM?

Pilihan	Skema bayar	Skala otomatis	Cocok untuk Gonanku?
Cloud Run 	Pay-per-request (Rp 0 saat idle)	Ya	YA — sempurna
App Engine Standard	Pay-per-instance hour	Lambat scale-up	Kurang ideal — minimum 1 instance kadang discharge
Compute Engine (VM)	Fixed ~Rp 150.000+/bulan	Manual	TIDAK — boros banget
GKE (Kubernetes)	\$74+/bulan untuk control plane	Ya	Overkill untuk app 1 instance

Kalimat untuk dosen:

"Pak, Cloud Run itu **serverless container**. Konsepnya sederhana: kalau aplikasi saya tidak ada user yang akses (idle), **instance/server-nya di-shutdown otomatis** dan **biaya = 0**. Begitu ada satu request masuk lagi, Cloud Run nyalakan instance baru dalam 1-2 detik (ini disebut 'cold start'), handle request, terus matikan lagi kalau idle.

Untuk Gonanku — yang paling cuma diakses saya, Bapak sebagai penguji, dan 1-2 orang saat demo — ini perfect. Tidak ada beban operasional saat tidak ada user. Total tagihan saya 6 hari pertama deploy: **Rp 1.029** — itu pun karena transfer data Asia Pacific yang memang lebih mahal dari US. Kalau dihitung setahun, kira-kira Rp 60.000 saja."

BAB 5: Setup Development di Laptop

5.1 Prasyarat (Tools yang Harus Ada di Laptop)

1. Python 3.11+ (cek: `python --version`)
2. Git
3. Editor: VS Code (recommended)
4. Terminal: PowerShell di Windows, atau Bash

5.2 Clone Repository

```
# Buka folder kerja
cd C:\Documents\KULIAH\SEMESTER 6\PROJECT

# Clone repo dari GitHub
git clone https://github.com/Sulthonikamalm/cloudtubes-gonanku.git PROJECTCLOUDTUBES

# Masuk ke folder project
cd PROJECTCLOUDTUBES
```

Penjelasan baris-per-baris:

- `cd ...` = pindah ke folder kerja kamu
- `git clone ...` = download seluruh kode dari GitHub ke laptop
- `cd PROJECTCLOUDTUBES` = masuk ke folder hasil clone

5.3 Setup Virtual Environment

```
# Buat virtual environment (semacam "kotak terisolasi" untuk install library)
python -m venv .venv

# Aktifkan environment
# Windows PowerShell:
.\.venv\Scripts\Activate.ps1
# Linux/Mac:
source .venv/bin/activate

# Install semua library yang dibutuhkan
pip install -r requirements.txt
```

Kenapa harus virtual environment?

Bayangkan kamu install library A versi 1.0 untuk Project X, dan Project Y butuh library A versi 2.0. Kalau install global, akan konflik. Virtual environment = kotak terpisah per project, library tidak bocor antar project.

5.4 Buat File `.env` (Konfigurasi)

```
# Copy template
copy .env.example .env

# Edit .env di VS Code → isi nilai-nilainya:
notepad .env
```

Isi `.env` yang minimal untuk development:

```
APP_NAME=Gonanku
APP_ENV=development
SECRET_KEY=development-key-acak-aja
DATABASE_URL= # kosong = otomatis pakai SQLite lokal
GROQ_API_KEY=gsk_XXXXXX # generate di https://console.groq.com/keys
GROQ_API_KEY_2=gsk_XXXXXX
GROQ_API_KEY_3=gsk_XXXXXX
TELEGRAM_BOT_TOKEN= # boleh kosong (upload akan fail, tapi UI bisa dia
TELEGRAM_CHAT_ID=
```

Note: `.env` masuk `.gitignore` — TIDAK di-push ke GitHub. Aman untuk simpan secret di sini.

5.5 Bikin Akun Pemilik Vault & Database

```
# Bikin database schema (lokal SQLite)
flask --app run db upgrade

# Bikin akun owner
flask --app run buat-pengguna sulthonika@gonanku.id "Sulthonika M" bungkersukses99
```

Output yang muncul:

```
Pengguna sulthonika@gonanku.id dibuat dengan 10 kategori default.
```

10 kategori default = Keuangan, Pendidikan, Pekerjaan, Kesehatan, Identitas, Properti, Kendaraan, Hobi, Hiburan, Lainnya. Bisa diedit nanti dari UI.

5.6 Run Aplikasi

```
python run.py
```

Output:

```
* Serving Flask app 'app'  
* Debug mode: on  
* Running on http://127.0.0.1:8080
```

Buka browser ke **<http://localhost:8080>**. Yang harus muncul:

1. Landing page (canvas cream, hero "Memorimu pantas disimpan dengan baik")
 2. Klik "**Masuk**" → halaman login
 3. Login pakai akun yang barusan dibuat → dashboard
-

BAB 6: Database & Migrasi Schema

6.1 Apa itu Migration?

Analogi:

Bayangkan database itu seperti rumah. Setiap kali kamu mau renovasi (tambah kamar, ubah cat dinding), kamu butuh **blueprint perubahan**. Migration = blueprint perubahan schema database.

Tools yang kita pakai: **Alembic** (di-wrap oleh Flask-Migrate). Setiap perubahan schema disimpan sebagai file Python di `migrations/versions/`.

6.2 Struktur File Migration

Di folder `migrations/versions/` ada beberapa file:

<code>d494b5cbd835_skema_awal_gonanku.py</code>	← migration #1: bikin semua tabel awal
<code>20b7543ede30_add_foto_profil.py</code>	← migration #2: tambah kolom foto_profil ke tal
<code>f3a8c2b71e09_foto_profil_ke_text.py</code>	← migration #3: ubah foto_profil dari VARCHAR()

6.3 Cara Kerja Migration

Scenario: Kamu ubah model `Pengguna` di kode Python, tambah kolom baru.

```
# 1. Generate file migration baru (Alembic deteksi perubahan dari model)
flask --app run db migrate -m "tambah kolom umur ke pengguna"

# 2. Cek file migration yang baru di-generate di migrations/versions/
# (Edit kalau perlu)

# 3. Apply ke database
flask --app run db upgrade
```

Alembic kerjanya begini:

1. Compare model SQLAlchemy Python dengan schema DB sekarang
2. Generate Python file dengan operasi `op.add_column(...)`, `op.alter_column(...)`, dll
3. Saat `db upgrade`, eksekusi operasi → ubah schema DB

6.4 Migration di Production vs Development

Environment	Database	Cara apply
Development (lokal)	SQLite di <code>instance/gonanku_dev.sqlite3</code>	<code>flask --app run db</code> <code>upgrade</code>
Production (Cloud Run)	Supabase PostgreSQL	<code>./migrate.sh</code> di Cloud Shell

PENTING: Apply migration ke Supabase **sebelum** deploy container baru ke Cloud Run. Kalau tidak, kode versi baru pakai schema versi lama → bug atau crash.

BAB 7: Fitur Utama

7.1 Authentication & Login

Flow Login

```
User input email + password di form login
→ Browser kirim POST /login dengan CSRF token
→ Server cek password pakai bcrypt hash
→ Kalau cocok: bikin session cookie (HTTPOnly, Secure, SameSite=Lax)
→ Redirect ke /dashboard
```

Kenapa pakai Flask-Login?

- Built-in session management
- `@login_required` decorator → satu baris untuk proteksi route
- `current_user` object untuk akses data user di template

Code snippet di `auth_routes.py` :

```
@auth_bp.route("/login", methods=["GET", "POST"])
@limiter.limit("5 per 5 minutes", methods=["POST"]) # rate limit anti-brute-force
def login():
    if request.method == "POST":
        email = request.form.get("email", "").strip().lower()
        kata_sandi = request.form.get("kata_sandi", "")
        pengguna = Pengguna.query.filter_by(email=email).first()
        if pengguna is None or not pengguna.cek_kata_sandi(kata_sandi):
            flash("Email atau kata sandi salah.", "bahaya")
            return render_template("login.html", email=email)
        login_user(pengguna)
        return redirect(url_for("dashboard.dashboard"))
    return render_template("login.html")
```

7.2 Upload Berkas — Aliran Detail

Flow upload single file:

1. User pilih file di form upload
2. Browser kirim POST /berkas/upload (single) atau POST /berkas/unggah-satu (AJAX per-file)
3. Server validasi:
 - Ekstensi file (whitelist: jpg, png, pdf, docx, dll)
 - Ukuran max 30 MB
 - Type signature (mencegah .exe rename jadi .jpg)
4. Server simpan file sementara di /uploads_temp/
5. Server kirim ke Telegram Bot API:
bot.sendDocument(chat_id=CHAT_ID, document=open(path, 'rb'))
6. Telegram balas: file_id, message_id, file_unique_id
7. Server simpan referensi ke DB tabel `berkas`:
 - telegram_chat_id, telegram_message_id, telegram_file_id
8. Server jalankan AI metadata:
 - Buat kode arsip (BRK-YYMMDD-XXX)
 - Tentukan tipe: foto / dokumen / video / audio
 - Ekstrak tanggal momen (EXIF foto, PDF metadata, atau dari nama file)
 - Untuk foto: panggil Groq Vision → dapat deskripsi
 - Untuk dokumen: ekstrak teks pakai PyPDF2 / python-docx
 - Panggil Groq text model → judul, kategori, tag, ringkasan, peringatan_privasi
9. Server simpan metadata ke DB
10. Hapus file sementara (free up /uploads_temp/)
11. Server return JSON {ok: true, judul, kode_arsip, url_detail}
12. Browser update progress bar

Bulk upload (15 foto / 10 dokumen):

- Browser kirim **1 request per file** ke /berkas/unggah-satu (AJAX)
- Tiap request kecil (< 32 MiB → lolos batas Cloud Run)
- User lihat progress per file di overlay modal

7.3 AI Metadata Otomatis

Apa yang Groq lakukan:

Input ke Groq text model:

```
Nama file: IMG_20220815_140312.jpg
Tipe: foto
Judul awal: (kosong)
Kategori tersedia: Keluarga, Pendidikan, Pekerjaan, ...
Isi (dari vision): "Foto outdoor di siang hari, sekelompok 5 orang
muda mengenakan toga wisuda warna hitam, salah satu memegang map
berijazah, latar belakang bangunan kampus."
```

Output Groq:

```
{
  "judul_ai": "Foto Wisuda Kakak di Telkom University Surabaya, Agustus 2022",
  "kategori_ai": "Keluarga",
  "tag_ai": ["wisuda", "kakak", "telkom", "surabaya", "agustus-2022"],
  "ringkasan_ai": "Foto kenangan wisuda kakak sarjana di Telkom University Surabaya pada",
  "peringatan_privasi": "",
  "tingkat_kepercayaan": 0.85
}
```

7.4 AI Vision untuk Foto

Bagaimana model multimodal bekerja:

1. Foto di-encode jadi base64 (string panjang)

2. Bungkus dalam payload JSON:

```
json { "model": "meta-llama/llama-4-scout-17b-16e-instruct", "messages": [{
  "role": "user", "content": [ {"type": "text", "text": "<system prompt deskripsikan
& OCR>"}, {"type": "image_url", "image_url": {"url": "data:image/jpeg;base64, ... "}}
] } }
```

3. Groq response: teks bahasa Indonesia, max 300 kata

4. Teks ini disimpan sebagai `teks_ekstraksi` di DB

5. Lalu di-feed ke Groq text model untuk generate metadata final

Kenapa 2 step (vision → text)?

- Vision model bagus deskripsikan gambar tapi tidak follow structured output sebagai text-only model
- Text model lebih jago return JSON valid

7.5 Chatbot Pencarian Natural

Flow:

```
User: "foto wisuda kakak tahun 2022 dong"
↓
Server panggil Groq dengan SISTEM_INTENT prompt
↓
Groq parse:
{
  "jenis_intent": "pencarian",
  "kata_kunci": ["wisuda", "kakak"],
  "tipe_file": "foto",
  "tanggal_mulai": "2022-01-01",
  "tanggal_selesai": "2022-12-31"
}
↓
Server query DB pakai filter ini → dapat kandidat 14 foto
↓
Server panggil Groq dengan SISTEM_RERANK prompt
  → AI re-rank semantik, pilih file yang relevan secara konsep
  → hasil: [id=5, id=23] saja (8 lainnya cuma kebetulan match keyword)
↓
Server panggil Groq dengan SISTEM_JAWABAN prompt
  → AI rangkai jawaban natural: "Aku menemukan 2 foto wisuda
    kakak – tertanggal Agustus 2022 di kampus Telkom University Surabaya."
↓
Server kirim JSON:
{
  "jawaban": " ... ",
  "berkas": [{id, judul, kode_arsip, url_detail}, ...]
}
↓
Browser render bubble chat + kartu file
```

Kenapa pakai semantic re-rank?

Tanpa re-rank: user cari "foto bermasker" → keyword "masker" match juga foto judul "foto pakai masker WiFi" (artinya masker WiFi alat). False positive.

Dengan re-rank: AI baca judul + ringkasan dari tiap kandidat, pilih yang **konsep**-nya cocok. Akurasi naik drastis.

7.6 Dashboard

Metrik yang ditampilkan:

- Total Arsip (semua aktif)
- Total Foto, Dokumen, Video, Audio
- Total Kategori aktif
- Tren Upload 30 hari terakhir (line chart SVG)
- Komposisi Tipe (donut chart)

- Kategori Terbanyak (top 5)
- Berkas Terbaru (5 file terakhir)

Kenapa data dinamis & real-time?

Setiap akses /dashboard, fungsi `ambil_ringkasan_dashboard(user_id)` query DB langsung. Tidak ada cache. Trade-off: load time ~200ms (acceptable), tapi data selalu fresh.

7.7 Manajemen Kategori & Tag

Kategori (one-to-many):

- 1 berkas → 1 kategori
- User bisa tambah/edit/hapus kategori
- Default 10 kategori saat akun dibuat

Tag (many-to-many):

- 1 berkas → banyak tag
- 1 tag → banyak berkas
- AI auto-suggest tag saat upload
- User bisa edit di halaman tag

7.8 Sampah & Hard Delete

Two-stage delete (mirip Trash di Google Drive):

Stage 1: Soft Delete

- Klik "Hapus" → set `dihapus_pada = sekarang`
- File tidak hilang, cuma tidak muncul di list utama
- File tetap di Telegram channel
- User bisa restore dari halaman /berkas/sampah

Stage 2: Hard Delete (irreversible)

- Klik "Hapus Permanen" di sampah → DB row deleted + file di Telegram di-delete
- Transaction safety:
 1. Try delete file di Telegram (best-effort, kalau gagal lanjut)
 2. Delete row di DB (cascade ke tag relations + log)
 3. Catat aktivitas "hapus_permanen"
- Return status: `{ok, telegram_ok, pesan}`

7.9 Activity Log

Setiap aksi user di-log:

- upload, edit, hapus, pulihkan, hapus_permanen, regenerasi_ai, login

Bisa dilihat di /aktivitas. User bisa hapus entry individual atau bersihkan semua.

7.10 Landing Page Premium

Kenapa harus ada landing?

- Pengunjung anonim (belum login) yang akses URL Cloud Run akan **bingung** kalau langsung kena halaman login
- Landing kasih konteks: "ini app apa, fungsinya apa, kenapa harus daftar"

Desain:

- **Inspirasi:** rampay.webflow.io (editorial, monochrome, big serif italic accent)
- **Palette:** canvas cream (#F4F1EA) + ink navy (#0F2854) — match dashboard
- **Tipografi:** Plus Jakarta Sans + Instrument Serif italic
- **NO purple gradient, NO glassmorphism orbs** (kena feedback awal: terlihat AI-generic)

Section:

1. Sticky navbar dengan glass effect + tombol "Masuk"
2. Hero: eyebrow + headline serif italic + CTA + mockup besar
3. Logo strip teknologi (Flask, Groq, Cloud Run, Supabase, Telegram)
4. Feature grid 3-kolom (Metadata, Vision, Chatbot)
5. Showcase row alternating (Dashboard, Chat, Vision dengan ilustrasi)
6. Stats band navy (5 API keys, 100% isolation, 30 MB, ∞ storage)
7. Steps timeline (Masuk → Upload → Cari)
8. CTA gradient
9. Footer

Parallax & Grid:

- Background pattern garis kotak 56×56px (line-soft 7% opacity)
 - Mask radial fade di tengah supaya konten tetap terbaca
 - Parallax scroll: grid +0.18, hero-stage -0.08, CTA grid +0.12
-

7.11 ★ CRUD Sistem — Operasi Lengkap (Bagian yang Diminta Dosen)

*Pak, ini bagian yang Bapak tanyakan: apakah sistem ada operasi CRUD lengkap.
Jawaban: YA, ada di 5 tabel/entitas. Berikut detail lengkapnya.*

Apa itu CRUD?

CRUD = Create, Read, Update, Delete — empat operasi dasar dalam manajemen data. Sebuah aplikasi yang punya CRUD lengkap berarti user bisa **menambah** data, **melihat** data, **mengubah** data, dan **menghapus** data.

Analogi sederhana: ibarat buku catatan, kamu bisa **menulis catatan baru** (Create), **membaca catatan** (Read), **mengoreksi catatan** (Update), dan **menghapus halaman catatan** (Delete).

Tabel yang Punya CRUD di Gonanku

#	Entitas (Tabel)	C	R	U	D	Halaman/Endpoint
1	Berkas (file arsip)	✓	✓	✓	✓	/berkas/*
2	Kategori	✓	✓	✓	✓	/kategori/*
3	Tag	✓	✓	✓	✓	/tag/*
4	Riwayat Chat	✓	✓	—	✓	/chat/*
5	Aktivitas (Log)	(otomatis)	✓	—	✓	/aktivitas/*
6	Pengguna (Profil)	(registrasi CLI)	✓	✓ (foto)	—	/profil/*

1 CRUD BERKAS (File Arsip) — Yang Paling Lengkap

Tabel: `berkas` di database Supabase PostgreSQL

Kolom utama: id, pengguna_id, kode_arsip, judul, nama_file_asli, tipe_file, ukuran_file, deskripsi, tanggal_momen, kategori_id, status_privasi, status_ai, ringkasan_ai, tag_ai, telegram_chat_id, telegram_message_id, tanggal_upload, dihapus_pada

+ CREATE — Tambah file baru

URL: `POST /berkas/upload` (form biasa) atau `POST /berkas/unggah-satu` (AJAX per-file)

Yang terjadi:

1. User pilih file di browser
2. Server validasi (ekstensi, ukuran maks 30 MB)

3. File dikirim ke Telegram channel
4. AI Groq baca isi file → kasih judul, kategori, tag, ringkasan
5. Semua metadata + referensi Telegram disimpan ke tabel `berkas`
6. Activity log dicatat: "upload"

Kalimat untuk Bapak:

"Operasi Create di sini bukan cuma insert row ke database, Pak. Ada 4 langkah: validasi, kirim ke Telegram, panggil AI, lalu simpan metadata. Jadi satu operasi Create berinteraksi dengan **3 layanan eksternal** sekaligus."

READ — Lihat file

URL:

- `GET /berkas/` — daftar semua file aktif dengan **filter & pagination**
- `GET /berkas/<id>` — detail satu file (judul, ringkasan AI, status, tag, log riwayat)
- `GET /berkas/sampah` — daftar file yang sudah dihapus (di sampah)

Filter yang bisa dipakai di list:

- Pencarian kata kunci (`q`) — cari di judul, deskripsi, kode arsip, ringkasan AI
- Filter kategori
- Filter tipe file (foto/dokumen/video/audio)
- Filter status privasi (normal/penting/sensitif/rahasia)

Kalimat untuk Bapak:

"Operasi Read di Gonanku tidak cuma 'tampilkan semua'. Ada 4 filter yang bisa dikombinasikan, plus pagination supaya tidak load 1000 row sekaligus. Plus operasi Read juga ada di chatbot — pakai natural language."

UPDATE — Edit metadata file

URL: `GET /berkas/<id>/edit` (form) → `POST /berkas/<id>/update` (submit)

Yang bisa diubah:

- Judul
- Deskripsi
- Tanggal momen
- Kategori
- Status privasi
- Tag (ditambah/dikurangi)

Plus operasi Update khusus: `POST /berkas/<id>/regenerasi-ai` — minta AI kerja ulang generate metadata baru.

Kalimat untuk Bapak:

"Update ada dua macam, Pak: edit metadata manual oleh user, dan regenerasi otomatis oleh AI. User bisa pilih, AI yang nulis judul (otomatis saat upload), atau user yang nulis sendiri (saat edit)."

DELETE — Hapus file (DUA TAHAP)

Gonanku punya konsep **soft delete + hard delete**, seperti Recycle Bin di Windows.

Tahap 1: Soft Delete — `POST /berkas/<id>/hapus`

- File **tidak benar-benar hilang**, cuma ditandai `dihapus_pada = sekarang`
- Hilang dari list utama, tapi masih di tabel
- File fisik di Telegram **masih ada**
- Bisa di-restore dari halaman Sampah

Tahap 2: Restore — `POST /berkas/<id>/pulihkan`

- Bersihkan tanda `dihapus_pada` → file muncul lagi di list utama

Tahap 3: Hard Delete — `POST /berkas/<id>/hapus-permanen`

- **Permanen**, tidak bisa dikembalikan
- Operasi: (1) hapus file dari Telegram channel, (2) hapus row di database, (3) cascade hapus relasi tag dan log

Tahap 4: Kosongkan Sampah — `POST /berkas/kosongkan-sampah`

- Hard delete **semua** file yang ada di sampah sekaligus

Kalimat untuk Bapak:

"Delete di Gonanku saya design 2 lapis, Pak. Soft delete dulu — supaya kalau user salah pencet, masih bisa kembalikan. Baru hard delete kalau yakin. Hard delete ini lebih kompleks karena harus juga hapus file fisik di Telegram, bukan cuma row database. Saya pakai pattern best-effort: kalau Telegram gagal hapus, database tetap clear, dan user dapat pesan warning."

2 CRUD KATEGORI

Tabel: `kategori`

Kolom: id, pengguna_id, nama

Default: saat akun dibuat, otomatis ada 10 kategori (Keuangan, Pendidikan, Pekerjaan, dll.)

Operasi	URL	Cara
Create	POST /kategori/tambah	Form di halaman kategori, input nama baru
Read	GET /kategori/	Tabel semua kategori + jumlah berkas per kategori
Update	POST /kategori/<id>/update	Edit nama langsung di tabel
Delete	POST /kategori/<id>/hapus	Hapus, file yang pakai kategori ini akan jadi "Lainnya"

Kalimat untuk Bapak:

"Pak, kategori juga lengkap CRUD-nya. Yang menarik: saat user hapus satu kategori, file yang pakai kategori itu tidak ikut terhapus — mereka otomatis dipindah ke kategori 'Lainnya'. Ini cascade behavior yang saya design di code, bukan default database."

3 CRUD TAG

Tabel: tag dan tabel relasi many-to-many berkas_tag

Kolom tag: id, pengguna_id, nama

Operasi	URL	Cara
Create	POST /tag/tambah	Form di halaman tag
Read	GET /tag/	Tabel tag + jumlah berkas yang pakai tag itu
Update	POST /tag/<id>/update	Rename tag
Delete	POST /tag/<id>/hapus	Hapus tag, otomatis hapus juga relasi di berkas_tag

Plus: tag juga otomatis ditambah oleh AI saat upload — tidak harus manual.

Kalimat untuk Bapak:

"Tag relasi many-to-many, Pak. Artinya satu file bisa punya banyak tag, satu tag bisa dipakai banyak file. Saat user hapus tag, saya pakai SQLAlchemy cascade — relasi berkas_tag ikut hapus otomatis. Tidak ada orphan record."

4 CRUD RIWAYAT CHAT

Tabel: riwayat_chat

Kolom: id, pengguna_id, pertanyaan, jawaban, daftar_berkas_relevan, dibuat_pada

Operasi	URL	Cara
Create	POST /chat/tanya	User submit pertanyaan, jawaban + ID file relevan disimpan
Read	GET /chat/riwayat/<id> (AJAX)	Klik item sidebar → load percakapan
Update	—	(Tidak ada — chat history sifatnya read-only)
Delete (per item)	POST /chat/<id>/hapus	Hapus 1 entry riwayat
Delete (semua)	POST /chat/bersihkan	Hapus semua riwayat user

Kalimat untuk Bapak:

"Riwayat chat punya CRD (tanpa Update), Pak. Logis — kalau user sudah tanya 'foto wisuda', tidak masuk akal di-edit nanti. User bisa hapus per item kalau ada riwayat yang sensitif, atau hapus semua untuk privacy."

5 CRUD AKTIVITAS (LOG)

Tabel: log_aktivitas

Kolom: id, pengguna_id, aksi, keterangan, berkas_id, dibuat_pada

Aksi yang dicatat: upload , edit , hapus , pulihkan , hapus_permanen , regenerasi_ai , login , tambah_kategori , dll.

Operasi	Cara
Create	Otomatis saat user lakukan aksi (background)
Read	GET /aktivitas/ — daftar log
Update	— (log immutable per audit best practice)
Delete (per entry)	POST /aktivitas/<id>/hapus
Delete (semua)	POST /aktivitas/bersihkan

Kalimat untuk Bapak:

"Pak, log aktivitas itu konsepnya audit trail — siapa, kapan, melakukan apa. Best practice: log harus immutable (tidak bisa di-edit), supaya valid sebagai bukti. Tapi user boleh hapus log mereka sendiri untuk privacy. Compromise: Read + Delete saja, no Update."

6 UPDATE PROFIL PENGGUNA

Tabel: `pengguna`

Kolom yang bisa diupdate: `foto_profil`

Operasi	URL	Cara
Create	(lewat CLI <code>flask buat-pengguna</code>)	Saat registrasi awal
Read	(otomatis di sidebar tiap halaman)	Tampilkan nama, email, foto
Update	<code>POST /profil/upload</code>	Klik foto profil di sidebar → upload baru
Delete	— (akun tidak bisa dihapus dari UI, by design — single-user vault)	

Kalimat untuk Bapak:

"Pak, foto profil khusus saya design simpan langsung di database sebagai **base64 data URL**, bukan sebagai file di filesystem. Kenapa? Karena Cloud Run filesystem-nya ephemeral — file di disk hilang setiap restart. Kalau saya simpan sebagai file, foto profil user hilang setiap 15 menit. Solusinya: resize ke 256x256, JPEG quality 80, encode base64, simpan di kolom TEXT database. Persistent forever."

Ringkasan CRUD Sistem (Untuk Slide PPT)

GONANKU — CRUD COMPLETENESS				
Entitas	Create	Read	Update	Delete
Berkas	✓	✓	✓	✓ (2-tahap)
Kategori	✓	✓	✓	✓
Tag	✓	✓	✓	✓
Riwayat Chat	✓	✓	—	✓
Log Aktivitas	✓	✓	—	✓
Profil	CLI	✓	✓	—

Total operasi CRUD: 18 endpoint

Total tabel terlibat: 7 (`pengguna`, `berkas`, `kategori`, `tag`, `berkas_tag`, `riwayat_chat`, `log_aktivitas`)

Apa yang Membuat CRUD Gonanku Spesial

1. **Bukan CRUD biasa** — setiap Create di Berkas memicu 3 service eksternal (Telegram, Groq, DB)
2. **Soft delete + Hard delete** — pattern profesional, bukan langsung hapus
3. **Cascade behavior** — hapus kategori tidak hapus file, tapi pindah ke "Lainnya"
4. **AI-augmented Create** — saat upload, AI otomatis isi judul, kategori, tag, ringkasan
5. **Audit trail otomatis** — setiap CRUD aksi dicatat di log_aktivitas
6. **Data isolation** — semua query WAJIB filter `pengguna_id` (User A tidak bisa lihat data User B)
7. **CSRF protected** — semua endpoint Update & Delete dilindungi token CSRF (security)

Kalimat penutup section CRUD untuk Bapak:

"Jadi Pak, kalau Bapak tanya 'mana CRUD-nya?' — jawabannya: ada di 5 entitas, total 18 endpoint, dengan pattern profesional (soft delete, cascade, audit trail). Bukan cuma 'add edit delete' biasa, tapi dengan integrasi 3 service eksternal dan AI sebagai augmentation. Saya rasa ini menjawab pertanyaan Bapak tentang CRUD."

BAB 8: Security Hardening

Ini bagian audit production-readiness. Saya lakukan ini di akhir pengembangan, sebelum deploy production.

8.1 CSRF Protection (Cross-Site Request Forgery)

Masalah tanpa CSRF:

Bayangkan kamu login Gonanku di tab 1. Lalu kamu buka website jahat di tab 2. Website jahat punya kode JavaScript:

```
html <form action="https://gonanku.app/berkas/123/hapus" method="post"></form>
<script>document.forms[0].submit()</script>
```

Browser otomatis kirim cookie session Gonanku ke request ini → file kamu kehapus tanpa kamu tahu.

Solusi: Flask-WTF CSRFProtect

1. Setiap session dapat **token unik** per render halaman
2. Setiap POST form HARUS sertakan token ini
3. Tanpa token valid → 400 Bad Request

Implementasi:

```
# app/extensions.py
from flask_wtf.csrf import CSRFProtect
csrf = CSRFProtect()
```

```
# app/__init__.py
csrf.init_app(app)
```

```
<!-- Setiap template POST form -->
<form method="post" action="...">
  <input type="hidden" name="csrf_token" value="{{ csrf_token() }}" />
  ...
</form>
```

```
<!-- layout.html: meta tag untuk AJAX -->
<meta name="csrf-token" content="{{ csrf_token() }}" />
```

```
// chat.js dan upload.js: baca dari meta tag
const CSRF_TOKEN = document.querySelector('meta[name="csrf-token"]').content;

fetch(url, {
  method: 'POST',
  headers: { 'X-CSRFToken': CSRF_TOKEN },
  body: ...
});
```

Verifikasi (saat audit):

- POST /login tanpa CSRF → **400** ✓ (ditolak)
- POST /login dengan CSRF → **302** ✓ (redirect ke dashboard)
- POST /logout tanpa CSRF → **400** ✓
- AJAX POST /chat/tanya tanpa header CSRF → **400** ✓
- AJAX POST dengan header → **200** ✓

8.2 Session Cookie Hardening

Tiga flag penting:

```
# config.py
SESSION_COOKIE_SECURE = True if APP_ENV=="production" else False
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SAMESITE = "Lax"
```

Penjelasan:

- **Secure**: cookie hanya dikirim via HTTPS (mencegah MITM di WiFi cafe)
- **HttpOnly**: JavaScript tidak bisa baca cookie (mencegah XSS curi sesi)
- **SameSite=Lax**: cookie tidak ikut request cross-origin POST (proteksi CSRF lapisan kedua)

8.3 Rate Limiting

Masalah: brute force password

Tanpa rate limit, attacker bisa POST /login 1000x per detik dengan password coba-coba sampai ketemu.

Solusi: Flask-Limiter

```
# app/extensions.py
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address
limiter = Limiter(key_func=get_remote_address)

# auth_routes.py
@auth_bp.route("/login", methods=["GET", "POST"])
@limiter.limit("5 per 5 minutes", methods=["POST"])
def login():
    ...
```

Verifikasi:

- POST /login 1-5x dengan password salah → 200 (ditolak, tapi bisa coba lagi)
- POST /login attempt #6 → **429 Too Many Requests** ✅
- GET /login tetap 200 (cuma POST yang dibatasi)

Storage:

- Dev: `memory://` (in-memory, hilang saat restart)
- Production Cloud Run single-instance: tetap `memory://` cukup
- Production multi-instance: pakai Redis (Memorystore)

8.4 SECRET_KEY Fail-Fast

Masalah:

Sebelumnya: kalau env `SECRET_KEY` tidak diset, kode fallback ke string default `"secret_dev_jangan_dipakai_di_production"`. Ini bahaya — kalau deployer lupa set, semua session token bisa dipalsukan attacker (karena string default-nya public di GitHub).

Solusi: refuse to boot

```
# config.py
def _ambil_secret_key():
    nilai = os.getenv("SECRET_KEY", "").strip()
    env = os.getenv("APP_ENV", "development").lower()
    if not nilai:
        if env in ("production", "prod"):
            raise RuntimeError(
                "SECRET_KEY wajib diset di environment produksi. "
                "Set env var SECRET_KEY ke string acak panjang (≥32 karakter)."
            )
        return _SECRET_KEY_DEV_FALLBACK
    return nilai
```

Hasil:

- Boot di dev tanpa SECRET_KEY → pakai fallback dev (OK)
- Boot di prod tanpa SECRET_KEY → **container refuse to start** ✓

8.5 Error Handlers Lengkap

Sebelumnya cuma handler 413 (request terlalu besar) dan 404. Sekarang:

Kode	Trigger	Pesan ke user
400	CSRF token expired/missing	"Sesi telah berakhir atau permintaan tidak valid. Silakan muat ulang halaman dan coba lagi."
404	URL tidak ada	"Halaman atau berkas tidak ditemukan."
413	Upload > batas	"Permintaan terlalu besar. Maks 30 MB per file..."
429	Rate limit hit	"Terlalu banyak percobaan. Silakan tunggu beberapa menit sebelum mencoba lagi."
500	Exception tak tertangani	"Terjadi gangguan di server. Tim Gonanku sudah dapat notifikasi." (log full traceback)

Kenapa handler 500 penting?

Tanpa handler, Flask default tampilkan halaman traceback panjang yang **bisa bocor info sensitif** ke user (path file, query DB, env vars).

8.6 ProxyFix untuk Cloud Run

Masalah:

Cloud Run terminate HTTPS di Load Balancer (LB), lalu forward request ke container kita pakai HTTP biasa. LB tambah header `X-Forwarded-Proto: https`. Tanpa ProxyFix:

- `request.is_secure` returns False (Flask kira request HTTP)
- `url_for(_external=True)` generate URL `http:// ...` bukan `https:// ...`
- `SESSION_COOKIE_SECURE` True tapi Flask internal anggap request unsecure → inkonsisten

Solusi:

```
# app/__init__.py
from werkzeug.middleware.proxy_fix import ProxyFix

if app.config.get("APP_ENV", "").lower() in ("production", "prod"):
    app.wsgi_app = ProxyFix(
        app.wsgi_app, x_for=1, x_proto=1, x_host=1, x_prefix=1
    )
```

`x_proto=1` artinya trust header `X-Forwarded-Proto` dari 1 hop proxy (Cloud Run LB).

BAB 9: Kegagalan & Cara Memperbaiki

INI SECTION PALING PENTING UNTUK PRESENTASI. Dosen akan respect kalau kita tunjukkan: "kita pernah gagal, ini cara kami solve".

9.1 KEGAGALAN: Cloud Run 32 MiB Limit (413 Request Entity Too Large)

Apa yang terjadi:

User upload 10 foto sekaligus, total 50 MB. Browser kirim 1 request multipart-form berisi 10 file. Cloud Run reject dengan **HTTP 413** sebelum request sampai ke aplikasi.

Kenapa terjadi:

*Cloud Run HTTP/1 punya limit **32 MiB per request**. Ini bukan setting kita, ini batas hard dari Cloud Run. Bulk upload 10 foto $\times$ 5 MB = 50 MB $\rightarrow$ tembus batas $\rightarrow$ ditolak.*

Cara solve (yang kami terapkan):

Refactor jadi per-file AJAX upload. Browser tidak lagi kirim 1 request besar, tapi **10 request kecil berurutan** ke endpoint baru `/berkas/unggah-satu`.

Implementasi:

- **Backend:** tambah route `POST /berkas/unggah-satu` yang terima 1 file + metadata
- **Frontend:** `upload.js` loop tiap file, tiap iterasi 1 `fetch()` ke endpoint
- **UX bonus:** user lihat progress per file, kalau 1 gagal yang lain tetap lanjut

Kalimat untuk dosen:

"Pak, di awal saya kira Cloud Run terima request sebesar apapun. Ternyata ada batas hard 32 MB per request. Saat saya coba upload 15 foto sekaligus yang totalnya lebih dari 32 MB, langsung muncul HTTP 413. Saya tidak bisa naikkan batas itu — itu peraturan Cloud Run sendiri.

*Jadi saya redesign: dari satu request besar dipecah jadi **request kecil per file pakai AJAX**. Browser kirim 15 request berurutan, masing-masing $<$ 32 MB, semua lolos. Bonus-nya: user bisa*

lihat progress per file — file mana yang sedang diupload, mana yang sudah selesai, mana yang gagal."

9.2 KEGAGALAN: PostgreSQL Strict ORDER BY dengan SELECT DISTINCT

Apa yang terjadi:

Di local SQLite query `SELECT DISTINCT ... ORDER BY computed_expression` berjalan. Di production Supabase PostgreSQL → **error**.

Pesan error:

```
SELECT DISTINCT, ORDER BY expressions must appear in select list
```

Kenapa terjadi:

SQLite permissive (toleran). PostgreSQL strict — kalau pakai `DISTINCT`, semua kolom di `ORDER BY` HARUS muncul di `SELECT`. Computed expression (mis. `COALESCE( ... )`) tidak match kalau tidak di-include.

Cara solve:

Buang `.distinct()` dari query, dedupe di Python.

```
# Sebelum (SQLite OK, Postgres FAIL)
hasil = query.distinct().order_by(...).all()

# Sesudah (kompatibel)
def _dedupe_preserve_order(items, key):
    seen, out = set(), []
    for x in items:
        k = key(x)
        if k not in seen:
            seen.add(k)
            out.append(x)
    return out

hasil = _dedupe_preserve_order(query.order_by(...).all(), key=lambda x: x.id)
```

Pelajaran:

"Selalu test di DB production yang sama. SQLite untuk dev cepat, tapi behavior beda. Sekarang saya kalau ada query kompleks selalu cek di Postgres juga."

9.3 KEGAGALAN: `.gcloudignore` Match `__init__.py`

Apa yang terjadi:

Build Docker sukses tapi container crash saat boot dengan error:

```
ModuleNotFoundError: No module named 'app'
```

Kenapa terjadi:

File `.gcloudignore` punya pattern `*.py` (maksud kami: exclude test file `_test_*.py`). Ternyata pattern ini juga match `__init__.py` (yang `__` mulai dengan `_`). Akibatnya Cloud Build skip semua `__init__.py` → Python tidak kenal `app` sebagai package.

Cara solve:

Hapus pattern itu. File test sudah di-gitignore, tidak akan ke-clone, jadi tidak perlu di-exclude lagi.

```
# .gcloudignore
- *.py
```

Pelajaran:

"Hati-hati dengan glob pattern. `*.py` lebih agresif dari yang dikira. Sekarang saya selalu test deploy `gcloud builds submit` lokal sebelum production."

9.4 KEGAGALAN: Timezone WIB Off-by-7 Hours

Apa yang terjadi:

Dashboard tampilkan "5 berkas diunggah hari ini" — padahal sebenarnya 7 file. Atau sebaliknya. Selalu off-by-some.

Kenapa terjadi:

Server Cloud Run pakai UTC. `datetime.utcnow()` adalah UTC. Tapi user di WIB (UTC+7). Kalau filter "hari ini" pakai `WHERE DATE(tanggal_upload) = CURRENT_DATE` → masalah:

- File upload jam 23:30 WIB = jam 16:30 UTC → counted as "today" UTC
- User cek jam 00:30 WIB = jam 17:30 UTC → query "today" UTC = beda hari → file hilang dari count

Cara solve:

```
from datetime import datetime, timedelta

def _awal_hari_wib_sebagai_utc():
    """Jam 00:00 WIB hari ini, dalam timezone UTC."""
    sekarang_utc = datetime.utcnow()
    sekarang_wib = sekarang_utc + timedelta(hours=7)
    awal_wib = sekarang_wib.replace(hour=0, minute=0, second=0, microsecond=0)
    return awal_wib - timedelta(hours=7) # convert back to UTC

# Query:
hari_ini_awal = _awal_hari_wib_sebagai_utc()
hari_ini_akhir = hari_ini_awal + timedelta(days=1)
Berkas.query.filter(
    Berkas.tanggal_upload ≥ hari_ini_awal,
    Berkas.tanggal_upload < hari_ini_akhir
)
```

9.5 KEGAGALAN: Chat History Hilang Saat Refresh

Apa yang terjadi:

User chat dengan bot, dapat hasil, lalu refresh browser → semua chat hilang. Frustrating.

Kenapa terjadi:

Awalnya kami simpan history di JavaScript state (variable). Browser refresh = state hilang. Tidak ada persistence.

Cara solve:

ChatGPT-style: sidebar history + on-demand AJAX loading.

1. Semua chat disimpan di DB tabel `riwayat_chat`
2. Sidebar kanan tampilkan list 50 riwayat terbaru

3. User klik 1 riwayat → AJAX `GET /chat/riwayat/<id>` → load pertanyaan + jawaban + kartu file
4. User klik "Chat Baru" → clear area pesan (DB tidak disentuh)
5. Submit pertanyaan baru → save ke DB → muncul di sidebar

Bonus:

- Sidebar pakai event delegation → item baru auto-handled
- "Hapus" per-item dengan double confirm
- "Hapus Semua" untuk bersih total

9.6 KEGAGALAN: "Foto Bermasker" Return Semua Foto

Apa yang terjadi:

User cari "foto bermasker" → chatbot return **20 foto**, padahal seharusnya cuma 2-3 yang relevan. False positive parah.

Kenapa terjadi:

Pencarian awal pakai keyword match SQL `LIKE %masker%` . Tapi:

- "Masker WiFi" → "masker" cocok tapi konteks tidak nyambung
- "Acara penggunaan masker COVID 2020" → "masker" cocok tapi user cari foto bukan flyer
- dst

Cara solve:

AI Semantic Re-rank — pattern 2-pass:

Pass 1: Keyword search SQL → dapat 20 kandidat (broad)

Pass 2: Kirim 20 kandidat ke Groq dengan prompt:

```
User cari: "foto bermasker"
Kandidat: id=1 "foto pakai masker WiFi", id=2 "selfie pakai masker N95",
          id=3 "foto wisuda kakak" ...
```

Pilih ID yang KONSEP-nya cocok dengan pertanyaan user.
Jangan asal keyword match.

Groq response: `ids_relevan: [2, 7]` (cuma 2 dari 20).

Pass 3: Filter SQL result hanya ID yang dipilih AI → return ke user.

Hasil:

Akurasi jump dari ~30% → ~85%. Trade-off: 1 ekstra Groq call per chat (latency +400ms). Worth it.

9.7 KEGAGALAN: Foto Profil Hilang Setiap Restart

Apa yang terjadi:

User upload foto profil di sidebar, foto tampil. Tunggu 15 menit (container idle, Cloud Run scale to zero). User akses ulang → **foto hilang**, sidebar tampilkan inisial nama lagi.

Kenapa terjadi:

*Saat itu kami simpan foto di `app/static/uploads/profil/profil_1_abc.jpg`. Cloud Run **filesystem ephemeral** — saat container shutdown, semua file `/tmp` dan folder app hilang.*

Cara solve (Opsi B yang kami pilih):

Simpan foto sebagai base64 data URL di kolom DB.

1. Ubah kolom `Pengguna.foto_profil` dari `VARCHAR(255)` → `TEXT` (migration f3a8c2b71e09)
2. Saat upload:

```
python img = Image.open(file_storage.stream) img = ImageOps.fit(img, (256, 256),
Image.LANCZOS) # crop square + resize buf = io.BytesIO() img.save(buf,
format="JPEG", quality=80, optimize=True) b64 =
base64.b64encode(buf.getvalue()).decode("ascii") data_url =
f"data:image/jpeg;base64,{b64}" pengguna.foto_profil = data_url # simpan langsung
di DB
```
3. Template render: `` (data URL langsung dipakai browser)

Trade-off:

- Row DB lebih besar (~10-25 KB per user)
- Untuk Gonanku (single-user vault, paling ratusan user) → OK
- Tidak perlu setup GCS / S3 → simpler

Pelajaran:

"Cloud Run filesystem ephemeral. JANGAN simpan apapun yang perlu persist ke /tmp atau folder static. Semua persistence harus ke DB atau external storage."

9.8 KEGAGALAN: File > 400 LOC (Project Rule)

Apa yang terjadi:

Audit kode menemukan 3 file melanggar rule project "max 400 baris per file":

- `layanan_groq.py` : **575 LOC**
- `layanan_berkas.py` : **428 LOC**

Kenapa rule ini ada:

File besar = sulit dibaca, sulit di-test, sulit di-review. Rule 400 LOC paksa kita modularisasi sebelum file membengkak.

Cara solve:

File 1: `layanan_groq.py` (575 → 342 LOC)

Strategi: ekstrak semua **system prompt string** (yang panjang, 200+ baris kalau ditotal) ke file terpisah.

- Bikin `app/services/groq_prompts.py` (271 LOC) — berisi 5 prompt: METADATA, INTENT, JAWABAN, RERANK, VISION
- `layanan_groq.py` import dari file baru
- Hasil: 342 LOC (logic) + 271 LOC (prompts) — keduanya di bawah 400

File 2: `layanan_berkas.py` (428 → 316 LOC)

Strategi: pindahkan **fungsi sampah** (soft delete, restore, hard delete, kosongkan) ke modul terpisah.

- Bikin `app/services/layanan_berkas_sampah.py` (155 LOC) — 5 fungsi sampah
- `layanan_berkas.py` re-export agar call site di routes tetap jalan tanpa modifikasi:

```
python from app.services.layanan_berkas_sampah import ( hapus_lunak_berkas, pulihkan_berkas, hapus_permanen_berkas, kosongkan_sampah, ambil_berkas_terhapus, )
```
- Hasil: 316 LOC + 155 LOC — keduanya di bawah 400

Pelajaran:

"Constraint kreatif itu bagus. Rule 400 LOC paksa saya bikin arsitektur yang lebih bersih. Re-export pattern juga memungkinkan split tanpa pecah API."

9.9 KEGAGALAN: Landing Page Versi Pertama Terlihat "AI-generated"

Apa yang terjadi:

Saya bikin landing pertama dengan palette purple gradient + glassmorphism floating orbs. Saat preview, **terlihat seperti template generik AI**.

Feedback dari kamu:

"Warna nya kok tidak sesuai dengan dashboard dan desainnya terlihat seperti vibe coding sekali. Saya tidak mau desain yang generik AI."

Kenapa terjadi:

Saya copy pattern Tailwind UI / Vercel template tanpa benar-benar studi inspirasi (Rampay). Hasilnya kombinasi cliché: ungu + biru gradient + blur orbs + glassmorphism card = "buatan AI di 2024".

Cara solve:

Total rewrite dengan vibe Rampay (rampay.webflow.io) yang asli:

- **Canvas warm off-white** #F4F1EA (bukan abu/biru tua)
- **Big serif italic accent** Instrument Serif untuk kata kunci ("Memorimu pantas *disimpan* dengan baik")
- **Monokromatik** navy match dashboard (#0F2854) — ZERO purple
- **Product-first hero** dengan mockup besar (browser chrome + sidebar simulasi + dashboard KPI + chart)
- **Editorial layout** dengan numbering serif italic 01/02/03
- **Generous whitespace** (section padding 100px+)

Plus iterasi lanjutan:

- Logo "G" placeholder → image logo-sm.png asli (3 spot)
- "kampus UGM" → "kampus Telkom University Surabaya" (sesuai kampus user)

- Grid pattern subtle ala Linear/Vercel + parallax scroll halus

Pelajaran:

"Inspirasi visual harus benar-benar dipelajari, bukan ditebak. Vibe AI-generic = kombinasi popular tools tanpa restraint. Editorial design = restraint + tipografi tegas + monokromatik."

BAB 10: Deploy ke Cloud Run

10.1 Prasyarat (One-Time Setup)

Akun & Project GCP

1. Buat akun Google Cloud (<https://console.cloud.google.com>) → claim **\$300 credit 90 hari pertama**
2. Buat project baru, namanya `gonanku-app`
3. Aktifkan layanan:
 - Cloud Run API
 - Cloud Build API
 - Artifact Registry API
4. Buat Artifact Registry repo:

```
bash gcloud artifacts repositories create gonanku-repo \ --repository-format=docker \ --location=asia-southeast1
```

Akun & Project Supabase

1. Buat akun Supabase (<https://supabase.com>) — free
2. New Project, namanya `gonanku` di region **Singapore**
3. Catat password DB-nya (sekali tampil saat create, hilang setelah)
4. Catat Connection String (Settings → Database → URI → Transaction mode 6543)

Akun Telegram

1. Buat private channel di Telegram (HP), namanya bebas
2. Chat **@BotFather** → `/newbot` → ikuti instruksi → dapat **Bot Token**
3. Add bot ke channel sebagai **admin** (penting!)
4. Kirim 1 pesan ke channel
5. Forward pesan itu ke **@userinfobot** → dapat **Channel ID** (format `-100xxxxxx`)

Akun Groq

1. Buat akun di <https://console.groq.com> — free
2. Bikin 3-5 API key di `/keys`
3. Catat semua key (`gsk_xxxx ...`)

10.2 Deploy Pertama Kali (Step-by-Step)

Step 1: Buka Cloud Shell

Klik icon `>_` di pojok kanan atas console GCP → terminal Cloud Shell terbuka.

```
# Pastikan project aktif benar
gcloud config get-value project
# Output: gonanku-app

# Kalau bukan, set:
gcloud config set project gonanku-app
```

Step 2: Clone Repo

```
git clone https://github.com/Sulthonikamalm/cloudtubes-gonanku.git
cd cloudtubes-gonanku
```

Step 3: Upload `.env.production`

Cara A: upload via UI Cloud Shell

1. Klik icon `:` di toolbar Cloud Shell → **Upload**
2. Pilih `.env.production` dari laptop
3. File masuk ke `~/ .env.production`
4. Pindahkan: `mv ~/ .env.production ~/cloudtubes-gonanku/.env.production`

Cara B: bikin manual

```
cp .env.production.example .env.production
nano .env.production # isi semua nilai
```

Step 4: Generate SECRET_KEY Baru (security best practice)

```
python3 -c "import secrets; print(secrets.token_hex(32))"
# Output: a900dc3f1dd7d16641c7586dfd10e2178ece41c0ff2f2fe4ca48db66a51150e2

# Replace di .env.production
sed -i 's|^SECRET_KEY=.*|SECRET_KEY=<paste-hasil-di-atas>|' .env.production
```

Step 5: Verifikasi `.env.production` Lengkap

```
grep -E "ganti_dengan|SUPABASE_PASSWORD|refxxxxx|isi_token_dari|^GROQ_API_KEY(_[23])?=gsk."
```

Output **harus kosong**. Kalau ada baris muncul, masih ada placeholder yang perlu diisi.

Step 6: Set Executable & Apply Migration

```
chmod +x migrate.sh deploy.sh  
  
./migrate.sh
```

Output sukses:

```
⇒ Gonanku Migration to Production DB  
⇒ Load env dari .env.production ...  
⇒ flask db upgrade (target: head) ...  
INFO Running upgrade 20b7543ede30 → f3a8c2b71e09, foto_profil ke Text  
=====
```

MIGRATION SELESAI

Step 7: Deploy

```
./deploy.sh
```

Akan prompt: `Sudah migrate? (y/N):` — ketik `y` + Enter.

Akan jalan 3 tahap:

- [1/3] Build image via Cloud Build (~3-5 menit)
- [2/3] Bangun env vars dari `.env.production`
- [3/3] Deploy ke Cloud Run

Output akhir:

```
=====
```

DEPLOY BERHASIL

```
=====
```

Service URL : `https://gonanku-app-xxx-as.a.run.app`

Step 8: Test di Browser Incognito

Buka URL Service di **incognito window** (Ctrl+Shift+N Chrome / Ctrl+Shift+P Firefox).

Yang harus terlihat:

1. **Landing page** (canvas cream, hero serif italic, mockup dashboard, grid pattern)
2. Klik "**Masuk**" kanan atas → halaman login
3. Login pakai akun yang sudah dibuat → dashboard
4. Test upload foto, chat, edit, hapus

10.3 Deploy Update (Setelah Code Change)

Saat kamu update kode di laptop, push ke GitHub, lalu deploy ulang:

```
# Di Cloud Shell
cd ~/cloudtubes-gonanku
git fetch origin main
git reset --hard origin/main

# Kalau ada migration baru:
./migrate.sh

# Deploy
./deploy.sh
```

BAB 11: Penjelasan Command Deploy Baris-per-Baris

Kamu kasih contoh command ini di pertanyaan. Saya jelaskan satu-per-satu:

```
cd ~/cloudtubes-gonanku
```

Artinya: Pindah ke folder repo Gonanku di home directory Cloud Shell.

Kenapa: Semua command setelah ini relatif ke folder repo.

```
git fetch origin main
git reset --hard origin/main
```

Artinya:

- `git fetch origin main` = download update terbaru dari GitHub branch `main` ke local Git data, tapi belum apply ke working files.
- `git reset --hard origin/main` = force overwrite working files ke versi yang barusan di-fetch, **discard semua perubahan lokal yang belum di-commit**.

Kenapa hard reset (bukan `git pull`)?

Di Cloud Shell kita mungkin punya perubahan iseng dari sesi sebelumnya (mis. nano edit yang gagal). `git pull` bisa stuck di conflict. `reset --hard` lebih agresif tapi pasti clean — semua file sama persis dengan GitHub.

```
TAG="v$(date +%Y%m%d%H%M%S)-exif"
```

Artinya: Buat variable `TAG` berisi timestamp + label. Contoh hasil: `v20260607164520-exif`.

Kenapa: Setiap image container butuh tag unik supaya Cloud Run bisa rollback ke versi sebelumnya kalau ada bug. Timestamp = unik & informatif (kapan deploy).

```
IMAGE="asia-southeast1-docker.pkg.dev/gonanku-app/gonanku-repo/gonanku:${TAG}"
```

Artinya: Buat variable `IMAGE` berisi path lengkap container image di Artifact Registry.

Format: `<region>-docker.pkg.dev/<project-id>/<repo>/<image-name>:<tag>`

Kenapa Artifact Registry, bukan Docker Hub? AR ada di same region dengan Cloud Run → pull image lebih cepat saat deploy + lebih murah (tidak ada egress cost).

```
gcloud builds submit --tag "${IMAGE}" --region=asia-southeast1
```

Artinya: Submit ke Cloud Build. Cloud Build akan:

1. Upload semua file di current dir ke server (sesuai `.gcloudignore`)
2. Build Docker image pakai Dockerfile
3. Push image ke Artifact Registry dengan tag `${IMAGE}`

Kenapa Cloud Build, bukan build lokal? Build lokal butuh Docker installed (Cloud Shell tidak punya). Cloud Build = build di-server, hasil langsung ke registry. ~3-5 menit untuk image Gonanku.

```
python3 -c "  
import json  
env = {}  
with open('.env.production') as f:  
    for line in f:  
        line = line.strip()  
        if not line or line.startswith('#') or '=' not in line:  
            continue  
        k, v = line.split('=', 1)  
        env[k.strip()] = v.strip()  
print(json.dumps(env))  
" > /tmp/env.yaml
```

Artinya: Convert `.env.production` (format key=value plain) jadi JSON, simpan ke `/tmp/env.yaml`.

Kenapa perlu convert? `gcloud run deploy --env-vars-file` terima format YAML (atau JSON karena JSON itu subset YAML 1.2). Plain `.env` tidak support escape karakter spesial seperti `@`, `:`, `/` di password.

Kenapa pakai Python? Bash regex untuk parse `.env` tricky kalau ada karakter spesial. Python lebih reliable.

```
gcloud run deploy gonanku-app \  
  --image "${IMAGE}" \  
  --region asia-southeast1 \  
  --memory 512Mi --cpu 1 \  
  --min-instances 0 --max-instances 1 \  
  --timeout 600 --port 8080 --concurrency 80 \  
  --allow-unauthenticated \  
  --env-vars-file /tmp/env.yaml
```

Penjelasan flag-flag:

Flag	Arti
<code>gonanku-app</code>	Nama service di Cloud Run (URL akan jadi <code>gonanku-app-xxx-as.a.run.app</code>)
<code>--image \${IMAGE}</code>	Pakai image yang barusan di-build
<code>--region asia-southeast1</code>	Singapore (closest ke Indonesia, latency ~30ms dari Surabaya)
<code>--memory 512Mi</code>	Allocate 512 MB RAM per instance. Gonanku idle ~100 MB, peak saat AI call ~300 MB. 512 MB cukup + buffer.
<code>--cpu 1</code>	1 vCPU per instance
<code>--min-instances 0</code>	Boleh scale to zero saat idle → tagihan = Rp 0 saat tidak ada user
<code>--max-instances 1</code>	Max 1 instance running bersamaan. Cukup untuk vault pribadi 1-2 user. Hemat dari billing surprise.
<code>--timeout 600</code>	Request timeout 600 detik = 10 menit. Penting buat bulk upload (15 foto × ~10 detik AI call = 150 detik).
<code>--port 8080</code>	Container expose port 8080 (sesuai Dockerfile)
<code>--concurrency 80</code>	1 instance handle max 80 request bersamaan
<code>--allow-unauthenticated</code>	URL Cloud Run boleh diakses tanpa Google login. Tanpa flag ini, butuh IAM token (susah untuk demo).
<code>--env-vars-file /tmp/env.yaml</code>	Inject semua env var dari file (SECRET_KEY, DATABASE_URL, dll)

BAB 12: 🎤 Persiapan Presentasi

12.1 Elevator Pitch (30 Detik)

"Selamat pagi Bapak. Saya Sulthonika. Hari ini saya presentasikan Gonanku — vault arsip pribadi yang cerdas.

Masalah: kita punya ribuan foto dan dokumen, tapi sulit menemukan kembali yang dibutuhkan karena nama file generik dan pencarian kita masih pakai keyword tepat.

Solusi Gonanku: AI membaca isi file dan otomatis kasih judul, kategori, tag. Pencarian pakai bahasa biasa — cukup tanya, AI rangkai ulang dan tarik file yang relevan.

Stack: Flask + PostgreSQL Supabase + Telegram untuk storage + Groq AI + deploy di Google Cloud Run. Total biaya operasional: Rp 1.029 selama 6 hari pertama deploy."

12.2 Demo Flow Live (5 Menit)

Persiapan:

1. Pastikan Cloud Run live (URL siap)
2. Buka 2 tab browser:
 - **Tab 1 (incognito)**: untuk demo landing → login
 - **Tab 2 (regular, sudah login)**: untuk demo fitur logged-in
3. Siapkan 3 file untuk upload demo:
 - 1 foto wisuda (untuk test AI Vision)
 - 1 PDF surat (untuk test ekstrak teks)
 - 1 screenshot chat (untuk test OCR)

Skrip Demo:

[0:00-0:30] Landing & Value Proposition

"Pak, ini halaman depan Gonanku. Saya design pakai vibe editorial — palette navy dan ice blue match dengan dashboard. Konten utama: hero serif italic, demo mockup interaktif, statistik teknis di band navy."

Scroll perlahan, tunjuk parallax effect.

[0:30-1:30] Login & Dashboard

"Klik 'Masuk' → halaman login dengan CSRF protection dan rate limit 5 attempt per 5 menit anti brute-force."

Login. Tunjukkan dashboard.

"Ini dashboard. Real-time: 1284 total arsip, 972 foto, 312 dokumen. Chart 30 hari terakhir SVG native — no library berat. Kategori terbanyak dan berkas terbaru."

[1:30-2:30] Upload + AI Metadata

"Sekarang saya demo upload. Drag file ke dropzone."

Drag 1 foto wisuda. Tunjukkan progress modal.

"File saya kirim ke Telegram private channel — gratis, unlimited storage. Sambil tunggu, AI Groq Vision baca isi foto, generate deskripsi. Lalu AI text generate judul, kategori, tag, ringkasan, peringatan privasi."

Tunggu finish. Buka detail.

"Lihat: AI generate judul 'Foto Wisuda Kakak di Telkom University Surabaya'. Tag: wisuda, kakak, telkom, surabaya. Ringkasan 2-3 kalimat. Semua otomatis — saya tidak ngetik apa-apa."

[2:30-3:30] Chatbot Pencarian Natural

"Ke menu Chatbot. Sekarang saya cari pakai bahasa biasa."

Ketik: "foto wisuda kakak tahun 2022"

"AI parse intent: kata kunci 'wisuda, kakak', tipe foto, tanggal 2022. Lalu re-rank semantik — AI pilih dari 14 kandidat keyword match, mana yang KONSEP-nya cocok. Hasil: 2 foto saja. Tanpa re-rank, kita dapat 14 false positive."

[3:30-4:30] Security Demo

"Saya tunjukkan security hardening. Open dev tools. Coba edit cookie session."

Tunjukkan cookie: HttpOnly = ✓, Secure = ✓, SameSite = Lax.

"HttpOnly: JavaScript tidak bisa baca → mencegah XSS curi sesi. Secure: cookie hanya via HTTPS. SameSite Lax: cookie tidak ikut request cross-origin POST → proteksi CSRF lapisan kedua."

Logout dengan klik logout. Coba POST /berkas/123/hapus dari Postman tanpa CSRF token: Response: 400 Bad Request. "CSRF token tidak valid".

[4:30-5:00] Closing

"Itu Gonanku. Total biaya: Rp 1.029/minggu. Stack: 100% serverless + free tier. Aplikasi production-ready dengan CSRF, rate limit, session hardening, ProxyFix Cloud Run, error handlers lengkap, dan 71 unit + integration test."

12.3 Slide Outline yang Disarankan

Kalau presentasi PowerPoint:

Slide	Konten
1	Title + nama + NIM + dosen pembimbing
2	Problem statement (3 masalah inti)
3	Solusi: Gonanku (logo + tagline)
4	Demo flow screenshot landing
5	Architecture diagram (BAB 3)
6	Stack technology + alasan ringkas
7	Kenapa Supabase, bukan Cloud SQL (perbandingan harga)
8	Kenapa Telegram untuk storage (free, kreatif)
9	Fitur utama (icon grid 8 fitur)
10	Security hardening checklist
11	3 Kegagalan & Solusi (lessons learned)
12	Demo live (atau video recording)
13	Roadmap & future work
14	Q&A + Thank You

BAB 13: ? Antisipasi Pertanyaan Dosen

Q1: "Kenapa pakai Telegram untuk storage? Apa tidak melanggar TOS Telegram?"

Jawaban:

"Telegram secara resmi mengizinkan bot mengirim dan mengelola file di channel/grup yang bot itu admin. Limit file 50 MB via Bot API adalah aturan publik mereka, dan tidak ada batas total storage. Untuk vault pribadi seperti Gonanku, ini perfectly compliant.

Yang dilarang adalah: spam, distribusi konten ilegal, eksploitasi commercial massive. Use case kami — personal vault — tidak masuk kategori itu."

Q2: "Bagaimana kalau Telegram bots ditutup oleh Telegram suatu saat?"

Jawaban:

"Risiko yang kami sadar. Mitigasi: arsitektur kami modular. Service `layanan_telegram.py` punya interface jelas — `kirim_berkas` , `hapus_pesan` , `buat_tautan` . Kalau Telegram tidak available, kami bisa swap ke GCS (Google Cloud Storage) atau S3 dengan ganti **1 file**, tidak rebuild app.

Tapi practical-nya, Telegram Bot API sudah 9 tahun stabil. Risiko sangat rendah."

Q3: "Kenapa tidak pakai Firebase untuk database + storage + auth sekaligus?"

Jawaban:

"Firebase NoSQL (Firestore) tidak cocok untuk Gonanku karena:

1. Query relational kompleks (filter kategori + tanggal + tipe + text search) → SQL jauh lebih ekspresif dari Firestore query
2. Firebase Auth bagus tapi vendor lock-in. Flask-Login + bcrypt → portable
3. Firebase Storage punya quota harian gratis tapi terbatas. Telegram unlimited.

Plus, secara akademik, saya ingin demo Cloud Computing dengan multiple service GCP — Cloud Run + Artifact Registry + Cloud Build. Firebase semua satu vendor, kurang nunjukkan integration."

Q4: "Apa bedanya Cloud Run dengan App Engine?"

Jawaban:

"Cloud Run vs App Engine:

- **Cloud Run**: serverless containers. Cocok untuk app dockerized. Scale to zero gratis. Pay-per-request.

- **App Engine Standard**: serverless tapi terbatas runtime tertentu (Python, Node, dll). Tidak bisa container custom. Cold start lebih lama.

- **App Engine Flexible**: terima container, tapi tidak bisa scale to zero — minimum 1 instance 24/7 → tagihan tetap jalan.

Cloud Run = sweet spot: fleksibilitas container + benefit serverless scale-to-zero."

Q5: "Bagaimana cara Gonanku skalabel kalau user 1000?"

Jawaban:

"Saat ini saya set `--max-instances 1` karena vault pribadi. Untuk multi-user 1000:

1. Naikkan `--max-instances` ke ~20
2. Ganti Supabase free tier (60 connection) → Pro tier (\$25/bulan, 200 connection)
3. Rate limiter backend dari memory → Redis (Memorystore)
4. Telegram bot mungkin perlu multiple bots untuk avoid rate limit (bot single 30 req/sec)
5. Tambah CDN (Cloud CDN) untuk static asset

Tapi untuk skala ini, Cloud Run handle 1000 user concurrent tanpa code change — cuma config."

Q6: "Kalau Groq down, apa terjadi?"

Jawaban:

"Sistem tetap jalan dengan degraded experience:

- Upload tetap sukses (file masuk Telegram + DB), tapi `status_ai = 'gagal'`
- User bisa tetap manual ngisi judul/kategori/tag
- Chatbot fallback ke keyword search SQL pakai kata kunci di pertanyaan (akurasi turun, tapi

tetap usable)

- User bisa retry regenerasi AI dari halaman detail file kapanpun.

Filosofi: AI augment, tidak block. Failover graceful."

Q7: "Bagaimana kalau ada bug critical setelah deploy?"

Jawaban:

"Cloud Run punya **revision system**. Setiap deploy menghasilkan revision baru — revision lama tetap tersimpan. Rollback satu command:

```
gcloud run services update-traffic gonanku-app --to-revisions=gonanku-app-00012-abc=100
```

Plus, kami punya error handler 500 yang log ke Cloud Logging. Saya monitor real-time via:

```
gcloud run services logs tail gonanku-app "
```

Q8: "Berapa biaya operasional sebulan?"

Jawaban:

"Berdasarkan billing report saya:

- Cloud Run + egress: ~Rp 1.000 / 6 hari → ekstrapolasi Rp 5.000/bulan
- Cloud Build (1 build = ~5 menit, 120 menit/bulan free tier) → Rp 0
- Artifact Registry storage (500 MB free) → Rp 0
- Supabase free tier → Rp 0
- Telegram → Rp 0
- Groq free tier → Rp 0

Total: ~Rp 5.000/bulan. Setahun ~Rp 60.000. Lebih murah dari kopi seminggu."

Q9: "Bagaimana isolasi data antar user? Buktikan!"

Jawaban:

"Setiap berkas, kategori, tag, riwayat chat punya kolom `pengguna_id` (foreign key ke `pengguna.id`). Semua query SQLAlchemy WAJIB filter `pengguna_id == current_user.id`.

Verifikasi yang sudah saya jalankan: bikin User A (saya) upload 50 file. Bikin User B (akun demo lain) login. Cek:

- GET /berkas/ → User B lihat list KOSONG ✓
- GET /berkas/1 (akses langsung URL file User A) → 404 ✓

- Chat 'tampilkan semua' di User B → AI return 'arsip kosong' ✓
 - Dashboard User B → total = 0 ✓
- Data User A dan User B fully isolated di level query."

Q10: "Apa rencana ke depan?"

Jawaban:

"Roadmap teknis:

1. **Semantic search dengan pgvector** — pakai Supabase pgvector extension, embed setiap berkas ke vector. Pencarian "foto sedih" bisa match foto melankolis walau judulnya tidak ada kata 'sedih'.
2. **Mobile app** — flutter atau PWA wrapper untuk akses dari HP
3. **Auto-import dari email/WhatsApp** — bot pull lampiran otomatis
4. **Collaboration** — share read-only vault ke keluarga (akses limited)
5. **Backup snapshot** — auto-export weekly ke Google Drive user

Roadmap bisnis: kalau scale up, freemium model. Free tier 100 file, Pro tier 50K file Rp 20.000/bulan."



Penutup

Gonanku adalah **bukti nyata** bahwa proyek skala produksi bisa dibangun dengan **budget Rp 0**, asal cermat pilih stack. Empat layanan eksternal — Supabase, Telegram, Groq, Cloud Run — semua punya free tier yang generous untuk use-case personal.

Tapi lebih dari teknologi, Gonanku adalah cerita tentang **proses iteratif**:

- Mulai dari ide sederhana
- Bertabrakan dengan masalah teknis (32 MiB limit, foto profil ephemeral, AI false positive)
- Belajar dari kegagalan
- Hardening untuk production (CSRF, rate limit, ProxyFix)
- Polish UX sampai detail (landing editorial, parallax subtle, logo asli)

"Pak, kalau ada satu kalimat yang saya pelajari dari project ini: **siap produksi itu bukan tentang kode yang sempurna, tapi tentang kode yang gagalpun masih bisa ditangani dengan baik**. CSRF gagal? Tampilkan pesan 400 yang ramah. Telegram down? User bisa coba upload lagi. AI down? User bisa input manual. Itulah pikiran engineering yang sesungguhnya."

Terima kasih.



Lampiran: Daftar Commit Penting (Bisa Ditunjuk ke Dosen)

```
05e2227 feat(landing): logo asli, kampus Telkom University Surabaya, grid + parallax
a50785e fix(deploy): ProxyFix middleware aktif di production untuk Cloud Run
3ee33b1 feat(security,refactor): rate-limit login, foto profil persistent, split layanan
915c34b chore(security,cleanup): production-readiness audit - CSRF, cookie hardening, sp
2900b3c refactor(landing): total rewrite - editorial vibe a la Rampay, hapus purple
7d52f6c feat(berkas): hapus permanen - bersihkan DB + file Telegram
1b9820a feat(upload): per-file AJAX upload + modal progress (fix 413 Cloud Run)
7c656c9 fix(ui): improve chatbot font readability in light and dark modes
7edea25 feat(metadata): auto-extract tanggal_momen dari EXIF/PDF/nama file
cc0cfa2 feat(chat): UX ChatGPT-style - fresh state + sidebar riwayat clickable
04ddc07 feat(chatbot): AI semantic re-rank menggantikan keyword match murni
```

Tunjukkan ke dosen: setiap commit ada cerita engineering decision-nya. Bukan asal commit.

Dokumen ini dibuat: 7 Juni 2026

Untuk: Tugas akhir Cloud Computing — Sulthonika Mulia / Telkom University Surabaya

Dosen pembimbing: Bapak (dosen penguji)